Guía de Actividades **Práctico-Experimentales** Nro. 004

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	- Freddy Ordoñez - Yimmy Angulo - Richard Guaman - Erick Jahirs - Joaquin Moscol
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Implementa algoritmos de abstracción mediante el diseño lógico y físico de Tipos de Datos Abstractos (TDA), asegurando eficiencia en la gestión de memoria.
Práctica Nro.	004
Título de la Práctica	Listas Dinámicas Simples sin Overhead y su Impacto en la CPU.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 22 de 10:30 – 11:30 Viernes 23 de mayo 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- **Diseñar e implementar** el Tipo de Dato Abstracto (TDA) Lista Simple Enlazada mediante la instanciación dinámica de nodos en JavaScript, evitando el uso de arreglos nativos.
- **Cuantificar y contrastar** el tiempo de ejecución T(n) entre la inserción de elementos al inicio de un arreglo dinámico (operación *unshift*) versus la inserción en la cabeza de una lista enlazada (operación *prepend*).
- **Evaluar** los principios de *Green Computing*, comprendiendo cómo la reducción de la complejidad temporal de *O(n)* a *O(1)* disminuye los

ciclos de reloj de la CPU, mitigando el consumo energético y térmico del hardware.

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Se aplica ABP para resolver un problema de consistencia de datos en la transferencia de pacientes entre médicos.

Contexto para el estudiante: Imagina que trabajas para el equipo de ingeniería de una nueva red social. El servidor debe procesar millones de peticiones de "siguiente video" por segundo. Si usas una estructura ineficiente, el servidor consumirá más electricidad, aumentará su temperatura y elevará los costos operativos.

Para resolver esto, implementaremos una Lista Doblemente Enlazada que permita saltar entre videos hacia adelante y hacia atrás sin tener que recorrer toda la lista desde el principio.

Tarea 1: Creación del TDA Nodo y Lista Simple

Empezaremos construyendo la unidad básica de memoria dinámica: el Nodo.

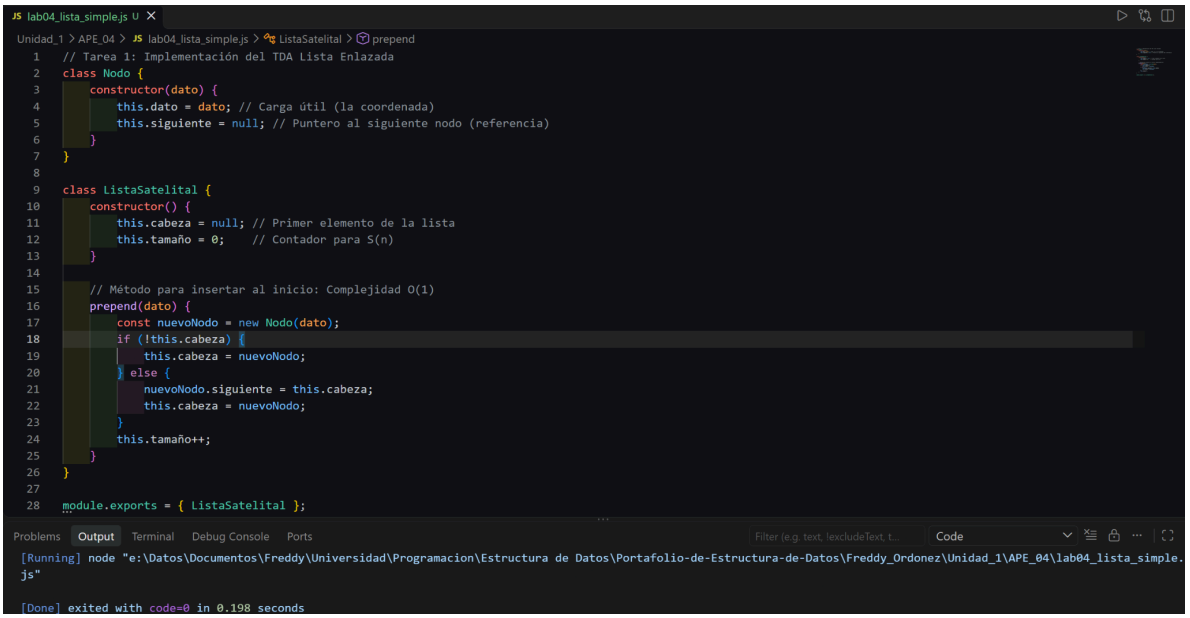
1. **En su directorio de trabajo, cree un archivo llamado lab04_lista_simple.js.**

2. **Implemente las clases Nodo y ListaSatelital utilizando clases modernas de JavaScript (ES6).**

```
// Tarea 1: Implementación del TDA Lista Enlazada
class Nodo {
    constructor(dato) {
        this.dato = dato; // Carga útil (la coordenada)
        this.siguiente = null; // Puntero al siguiente nodo (referencia)
    }
}

class ListaSatelital {
    constructor() {
        this.cabeza = null; // Primer elemento de la lista
        this.tamaño = 0;    // Contador para S(n)
    }

    // Método para insertar al inicio: Complejidad O(1)
    prepend(dato) {
        const nuevoNodo = new Nodo(dato);
        if (!this.cabeza) {
            this.cabeza = nuevoNodo;
        } else {
            nuevoNodo.siguiente = this.cabeza;
            this.cabeza = nuevoNodo;
        }
        this.tamaño++;
   module.exports = { ListaSatelital };
```



Tarea 2: Simulación del Estrés Térmico (Arreglo Nativo vs Lista Enlazada)

Vamos a demostrar empíricamente por qué la Lista Enlazada es una estructura de datos sostenible (Green Computing) frente a inserciones masivas.

1. **Cree un archivo llamado lab04_benchmark.js e importe la clase creada.**
2. **Implemente un script que mida los milisegundos requeridos para insertar N = 100,000$coordenadas al inicio de ambas estructuras**.

```
const { performance } = require('perf_hooks');
const { ListaSatelital } = require('./lab04_lista_simple');

const N = 100000; // 100 mil registros satelitales

console.log(`[Benchmark] Iniciando inserción de ${N} elementos al INICIO...`);

// ------------------------------------------------------
// PRUEBA 1: ARREGLO NATIVO (Desplazamiento de memoria O(n))
// ------------------------------------------------------
let arregloHistorial = [];
const startArray = performance.now();

for (let i = 0; i < N; i++) {
    // unshift() fuerza a V8 a mover los índices de todos los elementos previos
    arregloHistorial.unshift({ id: i, lat: 10.5, lng: -79.2 });
}

const endArray = performance.now();
const timeArray = (endArray - startArray).toFixed(2);
console.log(`[Energía Alta] Arreglo nativo (unshift): ${timeArray} ms`);

// ------------------------------------------------------
// PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
// ------------------------------------------------------
let listaHistorial = new ListaSatelital();
const startList = performance.now();

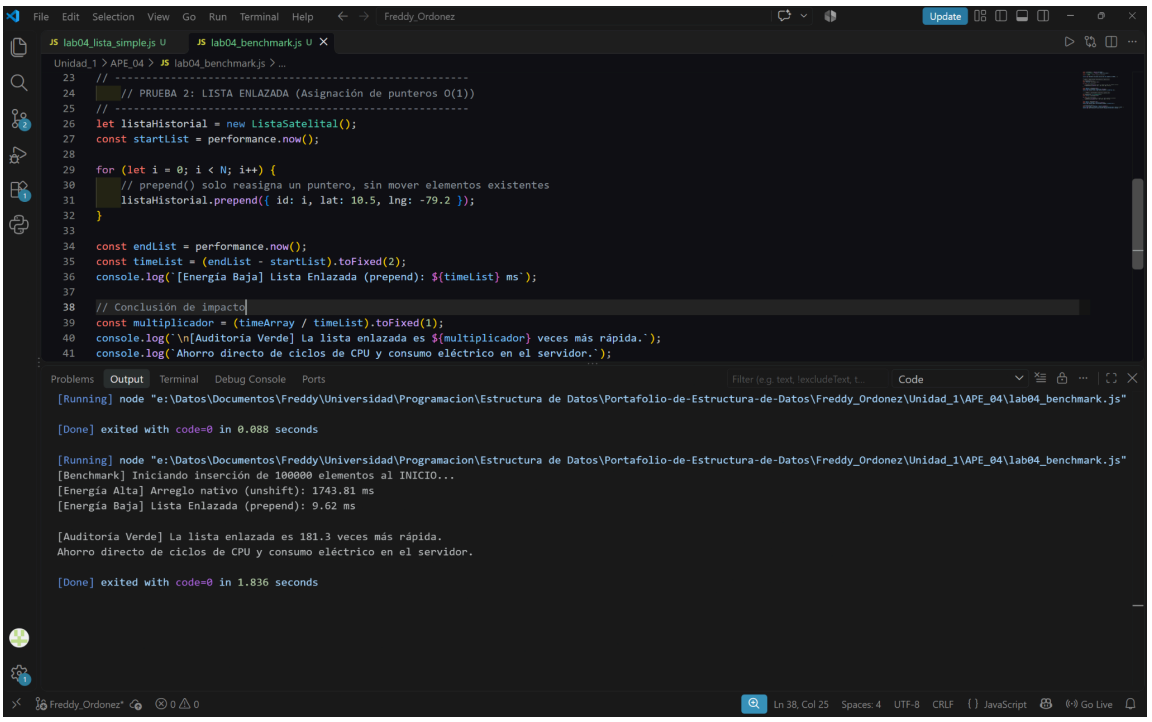
for (let i = 0; i < N; i++) {
    // prepend() solo reasigna un puntero, sin mover elementos existentes
    listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
}

const endList = performance.now();
const timeList = (endList - startList).toFixed(2);
```

```
console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);

// Conclusión de impacto
const multiplicador = (timeArray / timeList).toFixed(1);
console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces más rápida.`);
console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.`);
```

3. **Ejecute el script con node lab04_benchmark.js.**



4. **Análisis: Documente los tiempos arrojados. Notará que la diferencia es abismal. Mientras que el arreglo presenta una degradación asintótica O(n^2) en el bucle completo, la lista mantiene un crecimiento lineal O(n).**

   La diferencia en el rendimiento entre ambas estructuras al insertar elementos al inicio radica fundamentalmente en cómo gestionan la memoria RAM: mientras que un arreglo nativo requiere desplazar físicamente todos sus elementos existentes en bloques contiguos de memoria cada vez que se usa **unshift()** (lo que genera una degradación asintótica de $O(n^2)$ en un bucle completo), la lista enlazada simplemente asigna un puntero de referencia mediante su operación **prepend()** sin mover los nodos ya creados, manteniendo un crecimiento lineal eficiente de $O(n)$ que reduce drásticamente el consumo de ciclos de reloj de la CPU y el estrés térmico del hardware.

Tabla 1: Resultados del Benchmark de Inserción ($N = 100,000$ elementos al inicio)

Métrica / Parámetro	Arreglo Nativo (unshift)	Lista Enlazada Simple (prepend)	Diferencia / Impacto Real
Tiempo de Ejecución	1743.81 ms	9.62 ms	La lista es 1734.19 ms más rápida.
Factor de Aceleración	Estructura base (1x)	181.3 veces más rápida	Reducción drástica de la latencia en milisegundos.
Comportamiento Térmico / CPU	Energía Alta (Desplazamiento masivo de índices en memoria contigua).	Energía Baja (Asignación directa de punteros sin mover datos).	Ahorro directo de ciclos de CPU y menor estrés térmico.

Tarea 3: Implementación del Recorrido (Traversal)

A diferencia de un arreglo, no podemos acceder a un nodo de la lista mediante índices (ej. lista[5]). Debemos recorrer la lista.

1. **Regrese al archivo lab04_lista_simple.js y agregue el siguiente método dentro de la clase ListaSatelital.**

```
// Método para imprimir la lista (Recorrido secuencial O(n))
imprimirPrimeros(limite) {
    let actual = this.cabeza;
    let contador = 0;
    let salida = "";

    while (actual && contador < limite) {
        salida += `[ID: ${actual.dato.id}] -> `;
        actual = actual.siguiente;
        contador++;
```

```
    }
    console.log(salida + "NULL");
}
```

```
// Tarea 1 y Tarea 3: Implementación del TDA Lista Enlazada y Recorrido
class Nodo {
    constructor(dato) {
        this.dato = dato; // Carga útil (la coordenada)
        this.siguiente = null; // Puntero al siguiente nodo (referencia)
    }
}

class ListaSatelital {
    constructor() {
        this.cabeza = null; // Primer elemento de la lista
        this.tamaño = 0;    // Contador para S(n)
    }

    // Método para insertar al inicio: Complejidad O(1)
    prepend(dato) {
        const nuevoNodo = new Nodo(dato);
        if (!this.cabeza) {
            this.cabeza = nuevoNodo;
        } else {
            nuevoNodo.siguiente = this.cabeza;
            this.cabeza = nuevoNodo;
        }
        this.tamaño++;
    }

    // Tarea 3: Método para imprimir la lista (Recorrido secuencial O(n))
    imprimirPrimeros(limite) {
        let actual = this.cabeza;
        let contador = 0;
        let salida = "";
        while (actual && contador < limite) {
            salida += `[ID: ${actual.dato.id}] -> `;
            actual = actual.siguiente;
            contador++;
        }
        console.log(salida + "NULL");
    }
}

module.exports = { ListaSatelital };
```

lab04_lista_simple.js U | lab04_benchmark.js U

Unidad_1 > APE_04 > lab04_benchmark.js > ...

```
22
23  // ------------------------------------------------------
24      // PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
25  // ------------------------------------------------------
26  let listaHistorial = new ListaSatelital();
27  const startList = performance.now();
28
29  for (let i = 0; i < N; i++) {
30      // prepend() solo reasigna un puntero, sin mover elementos existentes
31      listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
32  }
33
34  const endList = performance.now();
35  const timeList = (endList - startList).toFixed(2);
36  console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);
37
38  // Conclusión de impacto
39  const multiplicador = (timeArray / timeList).toFixed(1);
40  console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces más rápida.`);
41  console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.`);
42
43  // ------------------------------------------------------
44  // Verificación de la Tarea 3 (Recorrido)
45  // ------------------------------------------------------
46  console.log("\n[Verificación Tarea 3] Primeros elementos de la Lista Enlazada:");
47  listaHistorial.imprimirPrimeros(5);
```

2. **Pruebe el método en su archivo benchmark.js añadiendo al final: listaHistorial.imprimirPrimeros(5); para verificar la correcta vinculación de los nodos.**

```
[Running] node "e:\Datos\Documentos\Freddy\Universidad\Programacion\Estructura de
js"
[Benchmark] Iniciando inserción de 100000 elementos al INICIO...
[Energía Alta] Arreglo nativo (unshift): 1746.67 ms
[Energía Baja] Lista Enlazada (prepend): 5.75 ms

[Auditoría Verde] La lista enlazada es 303.8 veces más rápida.
Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.

[Verificación Tarea 3] Primeros elementos de la Lista Enlazada:
[ID: 99999] -> [ID: 99998] -> [ID: 99997] -> [ID: 99996] -> [ID: 99995] -> NULL

[Done] exited with code=0 in 1.847 seconds
```

Como se observa en los resultados de la consola, el método de recorrido secuencial permite navegar exitosamente a través de los nodos de la lista enlazada desde la cabeza (**this.cabeza**) hasta el límite especificado. A diferencia de los arreglos estáticos, este recorrido confirma que la estructura no utiliza índices numéricos directos, sino que enlaza lógicamente cada elemento mediante referencias en memoria con una complejidad temporal de $O(n)$, garantizando la correcta vinculación de los datos satelitales tras el proceso de inserción masiva.

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

6. Resultados Esperados

El estudiante evidenciará a través del profiler (rendimiento en ms) que el uso de abstracciones de datos correctas mitiga drásticamente la latencia del software. Al reemplazar una estructura de datos de bloques contiguos (Arreglo) por una dispersa conectada por punteros (Lista Enlazada), se demuestra el ahorro de cálculos de la Unidad Aritmético Lógica (ALU) al evitar los redimensionamientos de memoria.

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

1. **A diferencia de un arreglo que pre-reserva una cantidad específica de Bytes contiguos, la Lista Enlazada nace como una referencia nula (null). Físicamente en la RAM, ¿de qué está hecha la "conexión" (el campo this.siguiente) que une a dos nodos esparcidos aleatoriamente por el *Heap*?**

   En la memoria RAM, la conexión representada por el campo **this.siguiente** no es un cable ni un bloque contiguo de bytes adyacente, sino una referencia de memoria o puntero como una dirección hexadecimal de 64 bits en sistemas modernos de 64 bit que almacena exactamente la dirección exacta en el *Heap* donde se encuentra alojado el siguiente objeto nodo en la memoria del sistema. Cada vez que se crea un nodo mediante la instanciación dinámica, el motor de JavaScript (como V8) le asigna un espacio arbitrario en el *Heap* y el campo **this.siguiente** guarda simplemente el valor numérico de esa dirección física, permitiendo que nodos completamente dispersos en la memoria interactúen de forma lógica.

   A nivel de hardware y bajo los principios de abstracción de software, esta dirección actúa como un puente lógico que la Unidad Aritmético Lógica (ALU) y el procesador siguen mediante saltos de punteros, evitando por completo la necesidad de desplazar físicamente los bloques de datos contiguos como ocurre en los arreglos nativos. De este modo, aunque los nodos estén distribuidos de manera caótica o aleatoria a lo largo de los chips de la memoria RAM, la referencia contenida en **this.siguiente** garantiza un encadenamiento estructurado que hace posible la navegación secuencial sin incurrir en el costo computacional de reorganizar la memoria física.

2. **Al invocar unshift() en un arreglo, el motor V8 es la causa eficiente que desplaza la memoria. En la operación prepend() de la Lista Enlazada, ¿cuál es el único agente o variable que se debe modificar para lograr la inserción exitosa sin desplazar datos?**

   El único agente que se debe modificar es el puntero de entrada de la lista, es decir, la referencia this.cabeza. Para insertar un nodo al inicio, primero se crea el nuevo nodo y se hace que su campo siguiente apunte a la cabeza actual (**nuevoNodo.siguiente = this.cabeza**). Luego se reasigna this.cabeza para que apunte al nuevo nodo. De todo lo que ya existía en la lista, solo cambia **this.cabeza.** Los demás nodos no se tocan, no se copian ni se reubican en la RAM.

   En el arreglo pasa lo contrario. Como sus elementos viven en un bloque contiguo y se acceden por índice, abrir un espacio en la posición 0 obliga a V8 a correr los n elementos una posición, lo que cuesta O(n). Repetido **N** veces, como en el benchmark con **N = 100,000**, el costo total crece de forma cuadrática. En la lista, **prepend()** hace una cantidad fija de asignaciones de referencias sin importar el tamaño, así que cuesta O(1), y el costo total de N inserciones es lineal, O(N).

   Desde la causa eficiente, el agente que produce el cambio ya no es el motor que reubica bloques de memoria, sino una sola reasignación de puntero hecha por nuestro propio código. El orden lógico de la lista se guarda en las referencias y no en la posición física de los datos. Por eso la inserción no necesita mover nada, y esto se traduce en menos ciclos de CPU, menos calor y menos consumo eléctrico, que es el argumento de Green Computing de la práctica.

3. **Si los arreglos nativos de JavaScript modernos están sumamente optimizados por lenguajes de bajo nivel (C++), ¿cuál es el verdadero telos (propósito final) de tomarse el tiempo para construir manualmente una estructura como la Lista Enlazada en entornos de producción?**

   El verdadero telos de construir una lista enlazada en JavaScript es adaptar la organización de los datos a las necesidades del problema, cuando sus características ofrecen una ventaja concreta. No se trata de superar automáticamente la optimización de los arreglos nativos.

   Una lista enlazada permite insertar o eliminar un nodo en O(1), siempre que ya se tenga la referencia al nodo y los enlaces necesarios. En cambio, insertar o eliminar al inicio o en medio de un arreglo suele requerir desplazar elementos. Esta propiedad puede ser útil, por ejemplo, en una caché LRU, donde una lista doblemente enlazada, combinada con un Map, permite localizar elementos y cambiar su posición de uso eficientemente.

Sin embargo, esa ventaja tiene un costo: cada nodo necesita referencias adicionales, genera trabajo para el recolector de basura y suele aprovechar peor la memoria caché. Además, buscar un elemento en la lista sigue siendo O(n). Por eso, en muchas aplicaciones, un arreglo resulta más sencillo y rápido en la práctica.

Desde la perspectiva de la causa final, el propósito no es construir una estructura por demostrar dominio técnico, sino satisfacer un patrón específico de operaciones con un costo adecuado. En producción, implementarla tiene sentido cuando el diseño y las mediciones justifican su uso; en el aprendizaje, su finalidad es comprender cómo las decisiones sobre memoria y enlaces determinan el comportamiento de una estructura.

4. **Un nodo que pierde la referencia (es decir, el nodo anterior ya no apunta a él) sigue existiendo físicamente en los transistores de la memoria RAM con su carga útil intacta, pero la Lista ya no lo reconoce. ¿Ha perdido ese nodo su esencia computacional? ¿Qué sucederá eventualmente con su existencia cuando despierte el *Garbage Collector*?**

   Sí, el nodo pierde totalmente su esencia computacional. Aunque físicamente sus datos sigan existiendo y ocupando espacio en la memoria RAM, lógicamente se vuelve un dato inútil. En la programación, la esencia o el propósito de un dato es poder ser leído o procesado, si la lista ya no lo reconoce porque cortamos su referencia puntero, es como si el dato dejará de existir para nuestro programa.

   Cuando el Recolector de Basura de JavaScript haga su recorrido por la memoria usando el algoritmo Mark-and-Sweep, se dará cuenta de que ninguna variable activa puede llegar hasta ese nodo. Al clasificarlo como basura inalcanzable, lo destruirá y liberará ese espacio físico en la RAM para que la computadora pueda reutilizarlo.

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

8. Bibliografía

Liste las fuentes utilizadas. Use normas **IEEE**. Pueden ser libros, manuales, artículos, tutoriales académicos, documentación oficial de software.

[1] B. Goetz, Java Concurrency in Practice. Addison-Wesley, 2022.

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.